

2069. Walking Robot Simulation II

Solved 

Medium

 Topics

 Companies

 Hint

A `width x height` grid is on an XY-plane with the **bottom-left** cell at `(0, 0)` and the **top-right** cell at `(width - 1, height - 1)`. The grid is aligned with the four cardinal directions (`"North"`, `"East"`, `"South"`, and `"West"`). A robot is **initially** at cell `(0, 0)` facing direction `"East"`.

The robot can be instructed to move for a specific number of **steps**. For each step, it does the following.

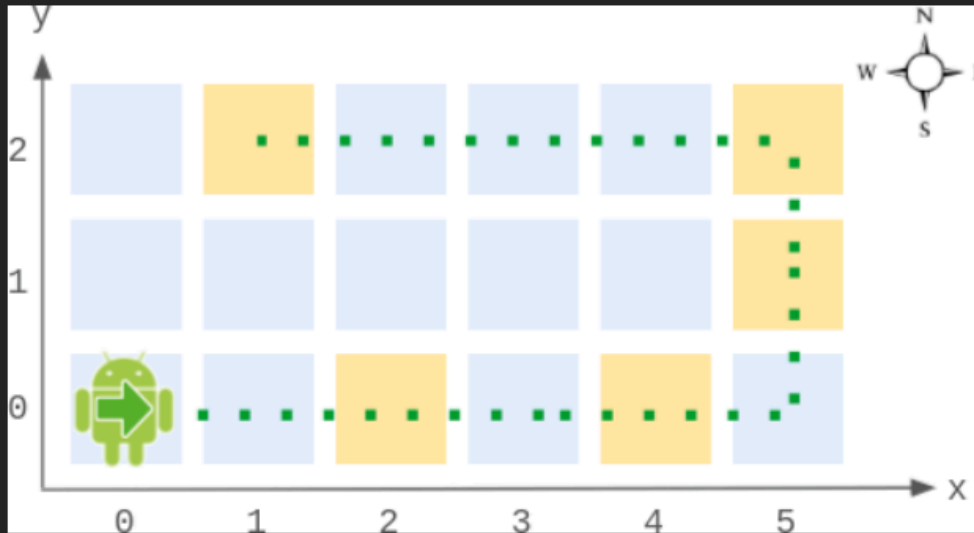
1. Attempts to move **forward one** cell in the direction it is facing.
2. If the cell the robot is **moving to** is **out of bounds**, the robot instead **turns** 90 degrees **counterclockwise** and retries the step.

After the robot finishes moving the number of steps required, it stops and awaits the next instruction.

Implement the `Robot` class:

- `Robot(int width, int height)` Initializes the `width x height` grid with the robot at `(0, 0)` facing `"East"`.
- `void step(int num)` Instructs the robot to move forward `num` steps.
- `int[] getPos()` Returns the current cell the robot is at, as an array of length 2, `[x, y]`.
- `String getDir()` Returns the current direction of the robot, `"North"`, `"East"`, `"South"`, or `"West"`.

Example 1:



Input

```
["Robot", "step", "step", "getPos", "getDir", "step", "step", "step",  
 "getPos", "getDir"]  
[[6, 3], [2], [2], [], [], [2], [1], [4], [], []]
```

Output

```
[null, null, null, [4, 0], "East", null, null, null, [1, 2], "West"]
```

Explanation

```
Robot robot = new Robot(6, 3); // Initialize the grid and the robot at (0,  
0) facing East.
```

```
robot.step(2); // It moves two steps East to (2, 0), and faces East.
```

```
robot.step(2); // It moves two steps East to (4, 0), and faces East.
```

```
robot.getPos(); // return [4, 0]
```

```
robot.getDir(); // return "East"
```

```
robot.step(2); // It moves one step East to (5, 0), and faces East.
```

```
                // Moving the next step East would be out of bounds, so it  
turns and faces North.
```

```
                // Then, it moves one step North to (5, 1), and faces
```

```
North.
```

```
robot.step(1); // It moves one step North to (5, 2), and faces North (not  
West).
```

```
robot.step(4); // Moving the next step North would be out of bounds, so it  
turns and faces West.
```

```
                // Then, it moves four steps West to (1, 2), and faces
```

```
West.
```

```
robot.getPos(); // return [1, 2]
```

```
robot.getDir(); // return "West"
```

Constraints:

- $2 \leq \text{width}, \text{height} \leq 100$
- $1 \leq \text{num} \leq 10^5$
- At most 10^4 calls **in total** will be made to `step`, `getPos`, and `getDir`.

Python:

class Robot:

```
def __init__(self, width: int, height: int):
    self.w = width
    self.h = height
    self.x = 0
    self.y = 0
    self.dir = 0 # 0=East, 1=North, 2=West, 3=South
    self.per = 2 * (self.w + self.h) - 4
```

```
def step(self, num: int) -> None:
```

```
    if self.per == 0:
        return
```

```
    num %= self.per
```

```
    # special case: full cycle
```

```
    if num == 0:
        if self.x == 0 and self.y == 0:
            self.dir = 3 # South
        return
```

```
    while num > 0:
```

```
        if self.dir == 0: # East
            move = min(num, self.w - 1 - self.x)
            self.x += move
            num -= move
            if num > 0:
                self.dir = 1
```

```
        elif self.dir == 1: # North
            move = min(num, self.h - 1 - self.y)
            self.y += move
```

```

        num -= move
        if num > 0:
            self.dir = 2

    elif self.dir == 2: # West
        move = min(num, self.x)
        self.x -= move
        num -= move
        if num > 0:
            self.dir = 3

    else: # South
        move = min(num, self.y)
        self.y -= move
        num -= move
        if num > 0:
            self.dir = 0

def getPos(self) -> List[int]:
    return [self.x, self.y]

def getDir(self) -> str:
    if self.dir == 0:
        return "East"
    elif self.dir == 1:
        return "North"
    elif self.dir == 2:
        return "West"
    else:
        return "South"

```

Your Robot object will be instantiated and called as such:

```

# obj = Robot(width, height)
# obj.step(num)
# param_2 = obj.getPos()
# param_3 = obj.getDir()

```

JavaScript:

// Added using AI

```

class Robot {
    constructor(width, height) {
        this.x = 0; this.y = 0;
        this.dir = "East" ;
    }
}

```

```

        this.width = width; this.height = height;
    }

    step(num) {
        const perim = 2 * (this.width - 1) + 2 * (this.height - 1);
        num %= perim;
        if (num === 0) num = perim;

        while (num > 0) {
            if (this.dir === "East") {
                const maxX = Math.min(this.x + num, this.width - 1);
                const rem = num - (maxX - this.x);
                num = rem;
                if (rem === 0) this.x = maxX;
                else { this.x = maxX; this.dir = "North"; }
            } else if (this.dir === "West") {
                const minX = Math.max(this.x - num, 0);
                const rem = num - (this.x - minX);
                num = rem;
                if (rem === 0) this.x = minX;
                else { this.x = minX; this.dir = "South"; }
            } else if (this.dir === "North") {
                const maxY = Math.min(this.y + num, this.height - 1);
                const rem = num - (maxY - this.y);
                num = rem;
                if (rem === 0) this.y = maxY;
                else { this.y = maxY; this.dir = "West"; }
            } else if (this.dir === "South") {
                const minY = Math.max(this.y - num, 0);
                const rem = num - (this.y - minY);
                num = rem;
                if (rem === 0) this.y = minY;
                else { this.y = minY; this.dir = "East"; }
            }
        }
    }

    getPos() { return [this.x, this.y]; }
    getDir() { return this.dir; }
}

```

Java:

```

class Robot {

```

```
int w, h;
int x, y;
int dir; // 0=East, 1=North, 2=West, 3=South
int per;
```

```
public Robot(int width, int height) {
    this.w = width;
    this.h = height;
    this.x = 0;
    this.y = 0;
    this.dir = 0; // initially East
    this.per = 2 * (w + h) - 4;
}
```

```
public void step(int num) {
    if (per == 0) return;

    num %= per;

    // special case: full cycle
    if (num == 0) {
        if (x == 0 && y == 0) {
            dir = 3; // South
        }
        return;
    }
}
```

```
while (num > 0) {
    if (dir == 0) { // East
        int move = Math.min(num, w - 1 - x);
        x += move;
        num -= move;
        if (num > 0) dir = 1;
    }
    else if (dir == 1) { // North
        int move = Math.min(num, h - 1 - y);
        y += move;
        num -= move;
        if (num > 0) dir = 2;
    }
    else if (dir == 2) { // West
        int move = Math.min(num, x);
        x -= move;
        num -= move;
    }
}
```

```

        if (num > 0) dir = 3;
    }
    else { // South
        int move = Math.min(num, y);
        y -= move;
        num -= move;
        if (num > 0) dir = 0;
    }
}

}

public int[] getPos() {
    return new int[]{x, y};
}

public String getDir() {
    if (dir == 0) return "East";
    if (dir == 1) return "North";
    if (dir == 2) return "West";
    return "South";
}
}

/**
 * Your Robot object will be instantiated and called as such:
 * Robot obj = new Robot(width, height);
 * obj.step(num);
 * int[] param_2 = obj.getPos();
 * String param_3 = obj.getDir();
 */

```